

Hands,On Tutorial: Getting Started with LLM,Based Process Control

This tutorial walks you through the existing work step by step. By the end you will understand how the system works, how to run the experiments, and where your contribution can begin.

Step 1: Thesis Overview

The system uses a fine,tuned language model as a controller for an industrial process. At every control step:

1. The process sends its current sensor readings
2. The LLM reads the readings and decides what actuators to activate
3. The decision passes through a safety check
4. The validated action is applied to the process
5. The loop repeats

The use case is the Festo MPS PA dual,tank workstation , a water machine with two tanks, a pump, a valve, and a heater. The two control targets are:

- Water level in the upper tank: 45% ($\pm 5\%$)
- Water temperature: 35°C ($\pm 3^\circ\text{C}$)

The framework is not specific to Festo. Any closed-loop system with sensors and actuators can use the same approach.

Step 2: Access the Festo Workstation

The physical Festo machine is located in the BTU lab. You need to be connected to the lab network , do not use university WiFi or VPN.

To monitor the machine:

- Open a browser and go to: http://192.168.25.25/get_data.html
- You see live sensor readings: tank level, temperature, valve states, pump state
- The page refreshes every 0.5 seconds

To manually control the machine:

- Open a browser and go to: http://192.168.25.25/post_data.html
- Use the buttons to turn the pump, valves, and heater on or off

Hands,On Tutorial: Getting Started with LLM,Based Process Control

To access system files:

- Open PuTTY, enter IP: 192.168.25.25, port: 22
- Login: username ubuntu
- Go to: `cd /var/www/html`
- The live sensor data file is at: /home/ubuntu/software_zedboard/json_data.txt
- This file is overwritten every 0.1 seconds with the latest readings

Built,in safety rules on the machine:

- Temperature above 38°C → heater forced off
- Left tank level above 70% → pump forced off
- Right tank minimum,level switch inactive → pump forced off

Step 3: Understand How Sensor Data Was Collected

The physical Festo machine was operated manually under different conditions , different pump speeds, valve positions, and heater states. A total of 3,714 sensor readings were recorded.

From these readings, the following physical rates were measured:

- How fast the tank fills at each pump power level
- How fast the tank drains when the valve opens
- How fast the water heats when the heater is on
- How fast the water cools when the heater is off
- When the float switch cuts off the pump
- How temperature behaves after the heater turns off (thermal inertia)

These measured rates are the foundation of the simulator.

Step 4: Run the Simulator

The simulator is written in C++. It replicates the physical behaviour of the Festo machine using the measured rates. It runs on a separate computer , not on the Festo machine itself.

Every 5 seconds the simulator does this:

1. Calculates new tank level and temperature based on current actuator states

Hands,On Tutorial: Getting Started with LLM,Based Process Control

2. Writes the current state to a shared JSON file
3. Reads the next actuator command from the shared file
4. Applies the command and advances one step

Why 5 seconds?

LLM inference takes 1–3 seconds. A 5,second interval gives the model enough time to respond before the next step.

To run the simulator, clone the repository:

```
git clone https://github.com/vidyashreerayar/festo,llm,process,control
```

Follow the setup instructions in the README to compile and run the C++ simulator.

Step 5: Understand the Training Data

The LLM does not learn from sensor readings alone. It learns from labeled control examples , each example shows the model what action to take in a given situation. Training examples were generated using fixed control rules applied across 14 operating scenarios. Each training example contains:

- Current tank level and temperature
- Last 6 control steps as history
- The correct actuator command to apply
- A short explanation of why that command is correct

A set of 500 correction examples is also included to teach the model how to recover from incorrectly formatted outputs.

Three datasets were created:

Dataset	Records	Predictive context	Notes
Experiment A	11,975	No	Baseline
Experiment B	9,000	Yes	More level,focused scenarios
Experiment C	3,000	Yes	Level and temperature balanced equally

Hands,On Tutorial: Getting Started with LLM,Based Process Control

All datasets are available in the repository.

Step 6: Load the Fine,Tuned Model

The base model is Llama 3.2,3B,Instruct. It is fine,tuned using QLoRA:

- Model weights are compressed to 4,bit to reduce memory usage
- Small trainable adapter layers (LoRA) are added on top
- Only the adapter layers are trained , about 30 million parameters out of 3.2 billion
- Training runs on a single GPU and needs only 3.4 GB of VRAM

Fine,tuned weights for all three experiments are available on Hugging Face:

<https://huggingface.co/vid1203>

To load a model, install the required libraries:

```
pip install transformers peft bitsandbytes trl
```

Then load the base model and attach the LoRA adapter for the experiment you want to run. The README in the repository contains the exact loading code.

Step 7: Understand the Prompt Format

At each control step, the Python supervisor builds a text prompt and sends it to the LLM. The prompt contains:

- The model's role and instructions (System Prompt)
- Current tank level and temperature
- Target values (45% level, 35°C temperature)
- Last 6 control steps as plain text key,value pairs
- Optionally: 3 predicted future states (if predictive context is enabled)

The LLM must return a strict 4,key JSON:

```
{  
  "pump_power": 70,  
  "upper_valve_open": false,
```

Hands,On Tutorial: Getting Started with LLM,Based Process Control

```
"heater_on": true,  
"reason": "Tank level is below target. Increasing the pump to fill faster."  
}
```

Four prompt strategies were tested:

Strategy	What it includes
Zero Prompt	Raw input only , no role, no reasoning, no examples
SP	System prompt defining model role and output format
SP + CoT	Also instructs model to reason step by step
SP + CoT + FS	Also provides 3 worked examples

Step 8: Understand the Predictive Context

In Experiments B and C, the LLM receives 3 predicted future states alongside the current state. The simulator calculates these predictions using the measured physical rates.

For example, if the tank is at 30% and the pump runs at 80%:

- Step +1: level 32%, temperature 32.3°C
- Step +2: level 34%, temperature 32.6°C
- Step +3: level 36%, temperature 32.9°C

These are added to the prompt as plain text. The LLM uses them to look ahead before deciding. Only the action for the current step is applied , the predictions are information only.

Step 9: Run a Closed,Loop Experiment

Start the simulator, then run the Python supervisor script. The full loop runs automatically:

1. Simulator writes current state to shared JSON file

Hands,On Tutorial: Getting Started with LLM,Based Process Control

2. Python script reads state and builds prompt
3. Prompt is sent to the fine,tuned LLM
4. LLM returns 4,key JSON
5. Safety fallback checks the response:
 - Temperature above 38°C → heater forced off
 - Tank level above 70% → pump forced off, valve opens
 - Pump and valve cannot both be active simultaneously
6. Validated action is written to shared file
7. Simulator reads action and advances one step
8. Repeat

A step counts as successful if:

- Level hit: upper tank within $\pm 5\%$ of 45%
- Temperature hit: water temperature within $\pm 3^\circ\text{C}$ of 35°C
- Overall: both conditions satisfied at the same time

Step 10: Check the Results

After the experiment runs, the evaluation script calculates:

Metric	What it measures
Level %	Steps where level is within $\pm 5\%$ of 45%
Temp %	Steps where temperature is within $\pm 3^\circ\text{C}$ of 35°C
Overall %	Steps where both are on target simultaneously
Format hallucination %	Steps where LLM returned invalid JSON
Safety override %	Steps where safety fallback had to intervene

Hands,On Tutorial: Getting Started with LLM,Based Process Control

Summary of existing results:

Exp	Prompt	Level%	Temp%	Overall%	Fmt H%	Safety Override%
A (12k)	SP+CoT+FS	65.4	85.9	60.0	0.0	0.0
B (9k)	SP+CoT+P	100.0	60.8	60.8	0.0	39.2
B (9k)	SP+CoT+FS+P	40.0	56.8	25.6	48.0	25.6
C (3k)	SP+CoT+P	84.7	67.8	57.6	0.0	32.2
C (3k)	SP+CoT+FS+P	86.2	61.2	52.6	0.0	38.8

What Is Already Done

The following is complete and publicly available:

- C++ simulator calibrated from real Festo sensor data
- Training datasets for all three experiments
- Fine,tuned LoRA adapters for all three experiments (huggingface.co/vid1203)
- Evaluation framework with all metrics
- Prompting templates for all four strategies
- Closed,loop results for all five configurations

Further Contribution:

The following areas are open and untested. Any one of these is a meaningful contribution:

1. PID baseline , A classical PID controller was never run on this system. Running PID on the same simulator and comparing its accuracy to the LLM results is the most direct open gap.
2. Physical hardware validation , All experiments ran inside the simulator. Running the same closed-loop evaluation on the actual physical Festo machine is untested and would validate whether the simulator results hold in reality.
3. Larger model zero,shot , Only Llama 3.2,3B was tested, and it required fine-tuning to work reliably. Testing a larger model such as QWEN 3 32B in

Hands,On Tutorial: Getting Started with LLM,Based Process Control

zero-shot mode (no fine-tuning) would answer whether model scale can replace fine-tuning.

4. History window size , The history window was always fixed at 6 steps (30 seconds). Testing shorter and longer windows would show how much history the LLM actually needs.
5. Serialization format , Sensor data was always passed as plain natural language text. Testing JSON, CSV, or table formats as prompt input would show which representation the LLM understands best.
6. Sampling rate , The control step was always 5 seconds. Testing at different sampling intervals would show how sensitive the system is to timing.

Useful Links

- Code and results: <https://github.com/vidyashreerayar/festo-llm-process-control>
- Fine,tuned model weights: <https://huggingface.co/vid1203>
- Festo live monitor: http://192.168.25.25/get_data.html
- Festo manual control: http://192.168.25.25/post_data.html